

IT ELECTIVE - SSO

Student Assignment Master Plan

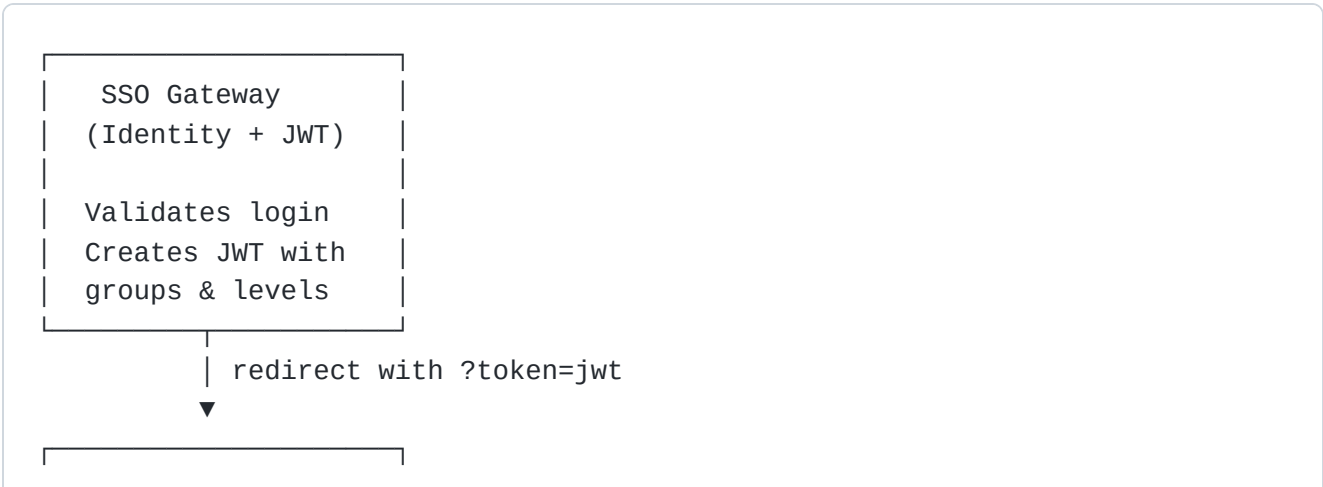
1. System Overview

A Centralized SSO Gateway that authenticates users once and redirects them to external client apps with JWT claims containing user info, groups, and access levels.

Key Decisions

Item	Decision
Tech Stack	ASP.NET Core MVC + EF Core SQLite
Identity	ASP.NET Core Identity (cookie-based)
Client Auth	CookieAuthenticationScheme + JWT
Token Lifetime	9 hours
Public Sign-ups	Disabled
Duplicate Emails	Blocked

2. Architecture



Client App (Any Stack)
PHP / Node / React / ASP.NET Core
Extracts JWT Creates session

3. Database Schema

Users (Identity)

- └─ Id (GUID)
- └─ Email (unique)
- └─ PasswordHash
- └─ IsActive (bool)
- └─ CreatedAt
- └─ LastLoginAt

TenantApps

- └─ Id (int)
- └─ Name (string, unique)
- └─ returnUrl (string)
- └─ IsActive (bool)
- └─ CreatedAt

Groups

- └─ Id (int)
- └─ TenantAppId (FK → TenantApps)
- └─ Name (string, e.g. "SalesApp-Admin")
- └─ PowerLevel (int, 0 = highest)
- └─ CreatedAt

UserGroups

- └─ UserId (FK → Users)
- └─ GroupId (FK → Groups)

AuditLogs

- └─ Id (int)
- └─ UserId (FK → Users, nullable)
- └─ Action (string)
- └─ Details (string)

```
|— IPAddress (string)
|— Timestamp (datetime)
```

4. JWT Claims Structure

```
{
  "sub": "user-id-guid",
  "email": "user@example.com",
  "tenant_app": "SalesApp",
  "groups": "SalesApp-Admin, SalesApp-Manager",
  "levels": {"SalesApp": 0, "SalesApp-Manager": 1},
  "exp": 1724170000
}
```

5. Issue List (19 Issues)

Epic 1: Foundation & Infrastructure

Issue 1: Project Scaffolding

Student: S1 | **Type:** Backend | **Priority:** High

As a developer, I want the solution structure created with all projects, dependencies, and configurations so that the team has a working foundation.

Tasks:

- ☐ Create solution IT ELECTIVE - SSO with 3 projects (Gateway, Data, Models)
- ☐ Add NuGet packages: EF Core, Identity, JWT Bearer, SQLite
- ☐ Configure appsettings.json with connection string and JWT settings
- ☐ Verify solution builds without errors
- ☐ Push to repo with proper .gitignore

Issue 2: Database Schema & EF Core Setup

Student: S1 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 1

As a developer, I want the EF Core DbContext and entity configurations created so that all team members can work against the same database schema.

Tasks:

- ☐ Create `SsoDbContext` inheriting from `IdentityDbContext`
- ☐ Define `TenantApp` entity with properties
- ☐ Define `Group` entity with FK to `TenantApp`
- ☐ Define `UserGroup` junction entity
- ☐ Define `AuditLog` entity
- ☐ Configure relationships in `OnModelCreating`
- ☐ Configure unique constraint on `TenantApp.Name`
- ☐ Configure unique constraint on `User.Email`
- ☐ Create initial migration
- ☐ Verify migration applies to SQLite

Issue 3: Day 1 Admin Seed Script

Student: S1 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 2

As the Platform Owner, I want a default admin account created automatically on first run so that the system is usable immediately.

Tasks:

- ☐ Add default admin credentials to `appsettings.json`
- ☐ Create `SeedData` static class
- ☐ Implement idempotent seed check (skip if admin exists)
- ☐ Create admin user via `UserManager`
- ☐ Ensure admin `IsActive = true`
- ☐ Call seed method in `Program.cs` on startup
- ☐ Write test: seed creates admin if missing
- ☐ Write test: seed skips if admin already exists

Epic 2: Admin API - User Management

Issue 4: User CRUD API

Student: S2 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 2

As the Main Admin, I want to create, list, view, and delete user accounts so that I can manage who has access to the system.

Tasks:

- ☐ Create UsersController (Admin area)
- ☐ GET /Admin/Users - list all users with pagination
- ☐ GET /Admin/Users/Create - show create form
- ☐ POST /Admin/Users/Create - create user with hashed password
- ☐ GET /Admin/Users/Details/{id} - show user details
- ☐ POST /Admin/Users/Delete/{id} - soft delete (set IsActive = false)
- ☐ Validate duplicate email before create
- ☐ Return appropriate error messages
- ☐ Write unit tests for user creation
- ☐ Write unit tests for duplicate email rejection

Issue 5: User Active Toggle

Student: S2 | **Type:** Backend + Frontend | **Priority:** High | **Depends on:** Issue 4

As the Main Admin, I want an On/Off toggle for each user so I can instantly pause or restore their access.

Tasks:

- ☐ Add POST /Admin/Users/ToggleActive/{id} endpoint
- ☐ Flip IsActive boolean on each call
- ☐ Add toggle switch UI to user list page
- ☐ Return JSON for AJAX toggle or redirect for form submit
- ☐ Verify login rejects inactive users (coordinate with S6)
- ☐ Show "Account Suspended" message on failed login
- ☐ Write test: toggle changes IsActive from true → false
- ☐ Write test: toggle changes IsActive from false → true

Issue 6: Admin Password Reset

Student: S2 | **Type:** Backend + Frontend | **Priority:** Medium | **Depends on:** Issue 4

As the Main Admin, I want to reset a user's password so that I can help users who are locked out.

Tasks:

- ☐ Add `POST /Admin/Users/ResetPassword/{id}` endpoint
- ☐ Generate random temporary password
- ☐ Hash and update password via `UserManager`
- ☐ Display temporary password to admin (one-time view)
- ☐ Force user to change password on next login (optional)
- ☐ Log password reset in `AuditLogs`
- ☐ Write test: reset password works
- ☐ Write test: audit log created on reset

Issue 7: Audit Logging

Student: S2 | **Type:** Backend | **Priority:** Medium | **Depends on:** Issue 2

As the Platform Owner, I want a log of all login attempts and admin actions so that I can review system activity.

Tasks:

- ☐ Create `AuditService` with methods: `LogLogin`, `LogAction`
- ☐ Log successful login (`UserId`, `Email`, `Timestamp`)
- ☐ Log failed login (`Email`, `Reason`, `IpAddress`, `Timestamp`)
- ☐ Log admin actions (create user, toggle active, reset password)
- ☐ Create `GET /Admin/AuditLogs` page to view logs
- ☐ Add pagination and date filtering
- ☐ Write test: audit log created on login
- ☐ Write test: audit log created on admin action

Epic 3: Admin API - Apps & Groups

Issue 8: Tenant App Registration

Student: S3 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 2

As the Main Admin, I want to register external apps so that only approved apps can use our login page.

Tasks:

- ☐ Create TenantAppsController (Admin area)
- ☐ GET /Admin/TenantApps - list all apps
- ☐ GET /Admin/TenantApps/Create - show create form
- ☐ POST /Admin/TenantApps/Create - save app name + return URL
- ☐ GET /Admin/TenantApps/Edit/{id} - edit form
- ☐ POST /Admin/TenantApps/Edit/{id} - update app
- ☐ POST /Admin/TenantApps/Delete/{id} - remove app
- ☐ Validate unique app name
- ☐ Validate return URL format
- ☐ Write test: create app works
- ☐ Write test: duplicate app name rejected

Issue 9: Return URL Validator

Student: S3 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 8

As the SSO system, I want to validate incoming return URLs so that only registered apps can use our login page.

Tasks:

- ☐ Create returnUrlValidator service
- ☐ Extract returnUrl from query string
- ☐ Match against TenantApps.ReturnUrl in database
- ☐ Return app details if valid, null if invalid
- ☐ Create error page for unapproved apps
- ☐ Display clear message: "Unapproved External App"

- ☐ Log invalid URL attempts in AuditLogs
- ☐ Write test: valid URL returns app
- ☐ Write test: invalid URL returns null
- ☐ Write test: unapproved URL logged

Issue 10: Group Management API

Student: S3 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 8

As the Main Admin, I want to create groups for specific apps with power levels so that external apps know user access levels.

Tasks:

- ☐ Create GroupsController (Admin area)
- ☐ GET /Admin/Groups - list all groups
- ☐ GET /Admin/Groups/Create - show create form with app dropdown
- ☐ POST /Admin/Groups/Create - auto-prefix [AppName] - [GroupName] , save level
- ☐ GET /Admin/Groups/Edit/{id} - edit form
- ☐ POST /Admin/Groups/Edit/{id} - update group
- ☐ POST /Admin/Groups/Delete/{id} - remove group
- ☐ Validate level (0 = highest, higher = less power)
- ☐ Ensure group name is unique per app
- ☐ Write test: group name auto-prefixed correctly
- ☐ Write test: level validation works

Issue 11: User-Group Assignment

Student: S3 | **Type:** Backend | **Priority:** High | **Depends on:** Issue 10

As the Main Admin, I want to assign users to groups across different apps so they get the right access for their job.

Tasks:

- ☐ Add POST /Admin/Users/{userId}/Groups - assign user to group
- ☐ Add DELETE /Admin/Users/{userId}/Groups/{groupId} - unassign
- ☐ Add GET /Admin/Users/{userId}/Groups - list user's groups

- ☐ Allow multiple groups per user (across different apps)
- ☐ Show assigned groups on user details page
- ☐ Write test: user can be assigned to multiple groups
- ☐ Write test: unassign removes relationship
- ☐ Write test: cannot duplicate assignment

Epic 4: Admin UI

Issue 12: Admin Dashboard & Navigation

Student: S4 | **Type:** Frontend | **Priority:** High | **Depends on:** Issue 1

As the Main Admin, I want a central dashboard with clear navigation so that I can efficiently manage users, apps, and groups.

Tasks:

- ☐ Create `_Layout.cshtml` for Admin area
- ☐ Add navigation sidebar: Dashboard, Users, Apps, Groups, Audit Logs
- ☐ Create `Dashboard` view with summary stats
- ☐ Style with Bootstrap
- ☐ Ensure responsive on mobile/tablet
- ☐ Add logout button
- ☐ Protect Admin area with `[Authorize]` + role check
- ☐ Verify regular users cannot access Admin area

Issue 13: User Management UI

Student: S4 | **Type:** Frontend | **Priority:** High | **Depends on:** Issue 4, 5

As the Main Admin, I want user-friendly pages to manage users so that I can perform admin tasks without technical knowledge.

Tasks:

- ☐ Build user list page with table (email, status, created date, actions)
- ☐ Add active toggle switch (AJAX or form submit)

- ☐ Build create user form (email, temporary password, confirm password)
- ☐ Build user details page (email, status, groups, last login)
- ☐ Add delete confirmation modal
- ☐ Show validation errors inline
- ☐ Add search/filter by email
- ☐ Write UI test: create user flow
- ☐ Write UI test: toggle active flow

Issue 14: Tenant App Management UI

Student: S4 | **Type:** Frontend | **Priority:** High | **Depends on:** Issue 8

As the Main Admin, I want a simple interface to register and manage external apps so that I can control who uses our SSO.

Tasks:

- ☐ Build app list page with table (name, return URL, status, actions)
- ☐ Build create app form (name, return URL)
- ☐ Build edit app form
- ☐ Add delete confirmation
- ☐ Show validation errors (unique name, valid URL)
- ☐ Add toggle to enable/disable app without deleting
- ☐ Write UI test: create app flow
- ☐ Write UI test: edit app flow

Issue 15: Password Reset UI

Student: S4 | **Type:** Frontend | **Priority:** Medium | **Depends on:** Issue 6

As the Main Admin, I want a simple button to reset a user's password and display the temporary password.

Tasks:

- ☐ Add "Reset Password" button to user details page
- ☐ Show confirmation dialog before reset
- ☐ Display temporary password in a copyable field

- ☐ Add "Copy to Clipboard" button
- ☐ Show success message after reset
- ☐ Log reset action in audit log

Epic 5: Auth Flow & Integration

Issue 16: Login Gateway & JWT Creation

Student: S6 | **Type:** Backend | **Priority:** Critical | **Depends on:** Issue 3, 5, 9

As a user, I want to log in on the central page and be sent back to my original app with my access claims attached.

Tasks:

- ☐ Create `AuthController` (public area, no auth required)
- ☐ GET `/Auth/Login?returnUrl=xxx` - show login form with hidden returnUrl
- ☐ Validate returnUrl against approved apps
- ☐ Show error if returnUrl invalid
- ☐ POST `/Auth/Login` - validate credentials via `SignInManager`
- ☐ Check `IsActive` flag
- ☐ Fetch user's groups and levels from database
- ☐ Create JWT with claims (sub, email, tenant_app, groups, levels, iat, exp)
- ☐ Redirect to `returnUrl?token=jwt`
- ☐ Handle failed login with clear error messages
- ☐ Implement rate limiting (5 attempts per 15 min)
- ☐ Log all login attempts to `AuditLogs`
- ☐ Configure CORS for external client apps
- ☐ Write test: valid login returns JWT
- ☐ Write test: invalid credentials rejected
- ☐ Write test: inactive account rejected
- ☐ Write test: invalid returnUrl blocked
- ☐ Write test: rate limiting works

Issue 17: Login Page Customization

Student: S6 | **Type:** Frontend | **Priority:** Critical | **Depends on:** Issue 16

As a user, I want a clean login page customized for our SSO so that I know I'm at the right place.

Tasks:

- ☐ Scaffold Identity login page
- ☐ Customize layout and branding
- ☐ Add SSO title "IT ELECTIVE - SSO"
- ☐ Display app name if provided in returnUrl
- ☐ Style error messages clearly
- ☐ Remove/lock down registration links
- ☐ Ensure form is accessible (labels, aria attributes)
- ☐ Write UI test: login flow
- ☐ Write UI test: error states display correctly

Issue 18: Mock Client App & Integration Test

Student: S6 | **Type:** Integration | **Priority:** High | **Depends on:** Issue 16

As a developer, I want a mock client app that receives and validates JWT so that we can test the full SSO flow end-to-end.

Tasks:

- ☐ Create minimal ASP.NET Core Web API project
- ☐ Implement JWT validation middleware
- ☐ Create GET /api/userinfo endpoint that reads claims
- ☐ Return user email, groups, and levels from JWT
- ☐ Add "Login with SSO" button that redirects to SSO
- ☐ Handle token expiry gracefully
- ☐ Create integration test: full flow
- ☐ Write README explaining how to run both apps together
- ☐ Document JWT claims structure

Issue 19: Integration Documentation

As a developer integrating with this SSO, I want clear documentation with working examples so that I can quickly implement authentication in my app regardless of tech stack.

Tasks:

Setup & Overview

- ☐ Create docs/ folder in repository
- ☐ Write docs/README.md - overview of SSO system
- ☐ Write docs/architecture.md - system diagram and flow
- ☐ Write docs/jwt-claims.md - claims structure with example payload
- ☐ Write docs/error-codes.md - all error messages and responses

Integration Guides

- ☐ Write docs/integration-aspnet-mvc.md - ASP.NET Core MVC client guide with full code examples:
 - Program.cs configuration (CookieAuthenticationScheme)
 - AccountController.cs (SsoLogin, Callback, Logout actions)
 - DashboardController.cs example (reading claims)
 - appsettings.json configuration
 - Flow diagram
- ☐ Write docs/integration-php.md - PHP client guide
- ☐ Write docs/integration-react.md - React client guide
- ☐ Write docs/integration-node.md - Node.js/Express client guide

Code Examples (Copy-Paste Ready)

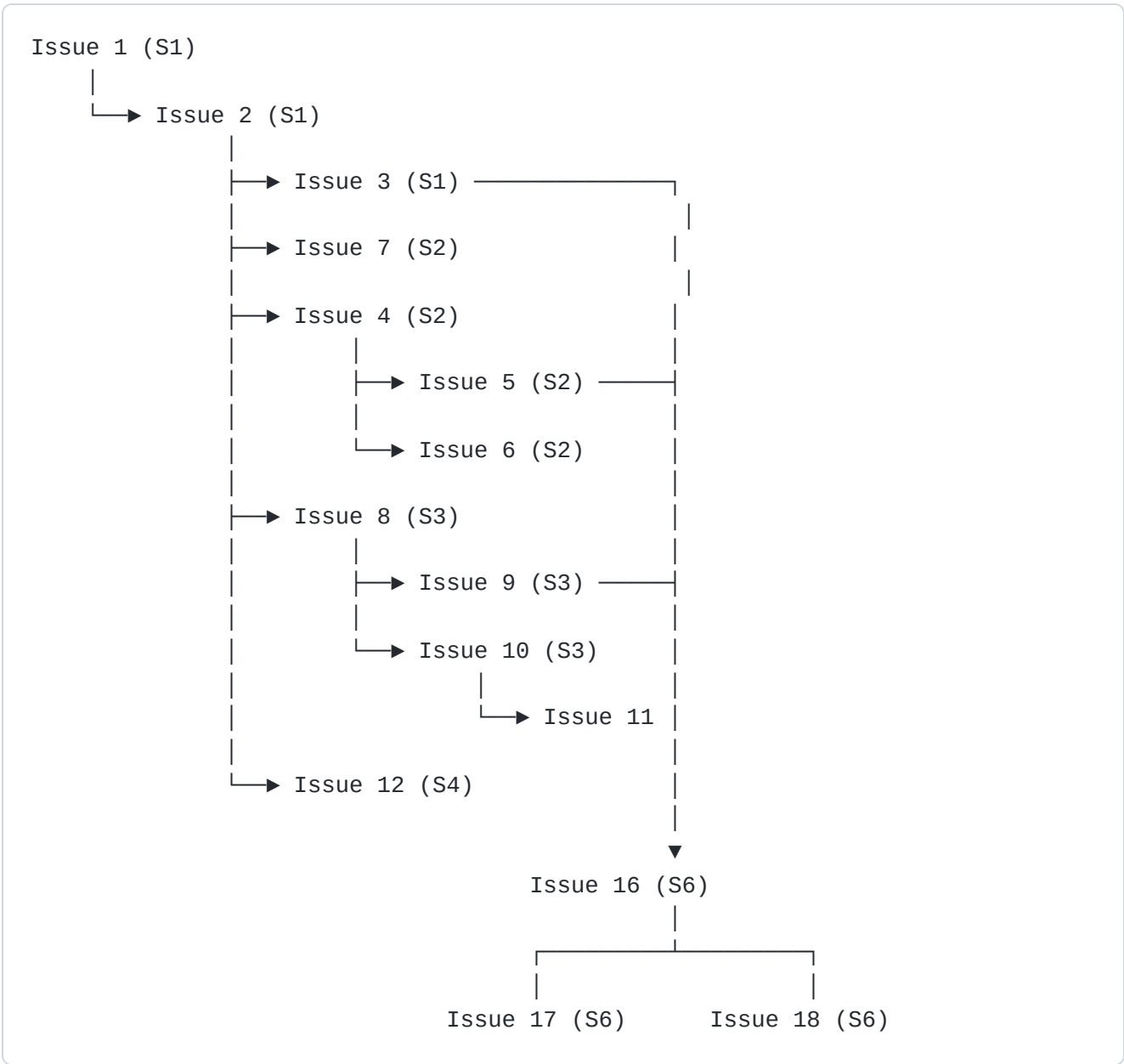
- ☐ Add PHP complete working example (config.php, index.php, callback handler)
- ☐ Add React complete working example (AuthContext, ProtectedRoute, Dashboard)
- ☐ Add Node.js/Express complete working example (middleware + callback route)
- ☐ Add ASP.NET Core MVC complete working example (Program.cs, AccountController, DashboardController)

Reference

- ☐ Document client app configuration (Sso:BaseUrl in appsettings.json)
- ☐ Document JWT validation steps (decode, verify expiry, extract claims)
- ☐ Document how to access user info from claims (email, groups, levels)

☐ Add FAQ section (common issues and solutions)

6. Dependency Map



7. Execution Phases

Phase	Week	Students	Issues
Foundation	1	S1 only	1, 2, 3
Backend APIs	2-3	S2, S3, S4	4, 5, 7, 8, 12

Phase	Week	Students	Issues
Feature Expansion	3-4	S2, S3, S4	5, 6, 9, 10, 14
Dependencies	4-5	S2, S3, S4	6, 11, 13
Auth Flow	5-6	S6 + S4	15, 16
Integration	6-7	S6, S5	17, 18, 19

8. Student Roles & Responsibilities

S1: Backend Foundation Engineer

Item	Details
Issues	1, 2, 3
Weeks	Week 1 only
Deliverables	Solution structure, database, seed script
Skills Required	C#, ASP.NET Core, EF Core, Git
Responsibilities	Create project scaffolding, define all entities, configure Identity, write seed script
Collaboration	Provides database schema that S2, S3, S4 depend on
Done When	<code>dotnet run</code> creates SQLite database with admin account

S2: User Management Engineer

Item	Details
Issues	4, 5, 6, 7
Weeks	Week 2-5
Deliverables	User CRUD, toggle, password reset, audit logging
Skills Required	C#, ASP.NET Core MVC, Identity, unit testing
Responsibilities	Build all user-related backend APIs and admin features

Item	Details
Collaboration	Works with S4 on UI, coordinates with S6 on login rejection
Done When	Admin can create users, toggle active, reset passwords, view audit logs

S3: Apps & Groups Engineer

Item	Details
Issues	8, 9, 10, 11
Weeks	Week 2-5
Deliverables	Tenant app CRUD, URL validator, group management, user-group assignment
Skills Required	C#, ASP.NET Core MVC, EF Core relationships, unit testing
Responsibilities	Build app registration, group management, and assignment logic
Collaboration	Works with S4 on UI, provides URL validator for S6's login flow
Done When	Admin can register apps, create groups, assign users to groups

S4: Frontend Engineer (Admin UI)

Item	Details
Issues	12, 13, 14, 15
Weeks	Week 2-6
Deliverables	Admin dashboard, all admin pages, user/app/group management UI
Skills Required	Razor Views, HTML, CSS, Bootstrap, JavaScript
Responsibilities	Build all admin-facing user interface pages
Collaboration	Depends on S2 and S3 for backend APIs, provides UI for S6 to reference

Item	Details
Done When	Admin can perform all tasks through browser without touching code

S5: Documentation & QA Engineer

Item	Details
Issues	19
Weeks	Week 5-7
Deliverables	Full documentation, integration guides, code examples
Skills Required	Technical writing, Markdown, understanding of all stack examples
Responsibilities	Write documentation, create integration examples for PHP/Node/React
Collaboration	Reviews S6's mock client, tests all flows, supports other students
Done When	External developer can integrate using only the documentation

S6: Auth Flow & Integration Engineer

Item	Details
Issues	16, 17, 18
Weeks	Week 5-7
Deliverables	Login gateway, JWT creation, login page, mock client app
Skills Required	C#, ASP.NET Core Identity, JWT, security concepts
Responsibilities	Build the core authentication flow that connects everything
Collaboration	Depends on S1 (seed), S2 (toggle), S3 (URL validator), works with S5 on docs

Item	Details
Done When	Full login flow works: user logs in → receives JWT → client app validates

9. Collaboration Matrix

	S1	S2	S3	S4	S5	S6
S1	-	Provides DB	Provides DB	Provides DB	-	Provides seed
S2	-	-	-	Provides API	-	Coordinates toggle
S3	-	-	-	Provides API	-	Provides URL validator
S4	-	Needs API	Needs API	-	-	-
S5	-	-	-	-	-	Needs flow
S6	Needs seed	Needs toggle	Needs validator	-	Needs docs	-

10. Weekly Check-in Agenda

Week	What to Review
1	S1: Solution builds, database created, admin seeded
2	S2, S3, S4: Starting their issues, any blockers
3	S2, S3, S4: Progress on backend APIs and UI
4	S2, S3, S4: Feature completion, integration issues
5	S6: Starting auth flow, S5: Starting documentation
6	S6: Login flow working, S5: Documentation draft

Week	What to Review
7	Final integration, demo, handoff

11. Git Workflow

```
main (protected - professor only)
├─ feature/issue-1-project-scaffolding
├─ feature/issue-2-db-schema
├─ feature/issue-3-seed-script
└─ ... one branch per issue
```

Process:

1. Student creates branch from `main`
2. Implements issue tasks
3. Opens PR → professor reviews
4. Merge to `main` after approval
5. Student picks next issue

12. Definition of Done (Per Issue)

- ☐ All tasks completed
- ☐ Code follows project conventions
- ☐ Unit/integration tests written and passing
- ☐ PR created with clear description
- ☐ PR reviewed and approved
- ☐ Merged to main

13. Summary

Metric	Value
Total Issues	19

Metric	Value
Total Tasks	181
Students	6
Timeline	7 weeks
Check-ins	Twice weekly
Face-to-face	1

All changes saved